

PROJECT INFONUM 59 - PROJECT REPORT

AI for Assisting in the Analysis of Financial Results

 [GitHub](#)

Céline HUDELLOT
Charles MOSLONKA
Hicham RANDRIANARIVO

Gabriel TRIER
Lucas TRAMONTE
Rayane BOUAITA

April 2025

Contents

1	Introduction	3
1.1	Context	3
1.2	Project Objectives and Deliverables	3
2	Project Organization	4
2.1	Task Planning and Monitoring	4
2.2	Tasks carried out	4
2.3	Collaborative Tools	5
3	Data Overview	5
3.1	User stories	6
3.1.1	Financial Summaries	6
3.1.2	ESG Practices Analysis	6
3.1.3	Investment & Portfolio Diversification	6
4	State of the Art	6
4.1	LLMs Finance	6
4.2	Domain Specialization	7
4.3	Data Enrichment	8
5	Technical Pipeline	9
5.1	Getting Started with LangChain	9
5.1.1	Retrieval-Augmented Generation (RAG)	9
5.1.2	First LangChain Integration	9
5.1.3	Improved LangChain version	10
5.2	RAG Vision approach	12
5.2.1	Challenges in RAG vision	12
5.2.2	ColPali	13
5.2.3	Indexing Phase	13
5.2.4	Retrieval Phase	15
5.2.5	Performances	16
5.2.6	Conclusion	17
6	RAG vision Pipeline Description	18
6.1	Pipeline Steps	18
7	Generative Models for Response Formulation	19
7.1	Google Gemma 3 Models	19
7.2	Qwen2 VL Models	19
7.3	Model Comparison Summary	20
8	Pipeline V2: Enhanced Pipeline with Re-ranking and Proposed Third Iteration	20
8.1	Limitations of Pipeline V1	20
8.2	Architecture and Enhancements of Pipeline V2	20
8.3	Optical Character Recognition and Preprocessing	21

8.4	Candidate Retrieval with Colpali	21
8.5	Detailed Analysis of the Cross Encoder Re-ranking Process	21
8.6	Recap and Future Directions	22
9	Dataset	22
9.1	Composition	22
9.2	Scope and Use Cases	22
10	Evaluation	23
10.1	Retriever Metrics	23
10.1.1	Revaluation ViDoRe	23
10.1.2	Retriever Metrics on Our Dataset	23
10.2	Generation Metrics	25
10.2.1	FinQA Evaluation	25
10.2.2	Generator Metrics on Our Dataset	26
10.2.3	Aggregation of Generation Metrics	28
11	Results and Analysis	29
12	Challenges Encountered	31
13	Conclusion project	31

1 Introduction

1.1 Context

This project is part of a collaboration between the [Artefact Research Center](#) and the [MICS Laboratory](#) at CentraleSupélec. The focus lies on generative AI, its applications, and the development of digital commons.

The project is aligned with the France 2030 initiative, supported by the French government, in collaboration with Mistral, Giskard, the INA, and the BnF. The overarching goal is to create digital commons for the French generative AI ecosystem. This endeavor aims to deliver applied, impactful, and shared research, particularly in the domain of large language models (LLMs), a topic experiencing significant growth and innovation.

1.2 Project Objectives and Deliverables

The aim of this project is the design and implementation of an end-to-end LLM-based agent. This includes the following components:

- **Data Preparation and Enrichment:** Preparing, qualifying, and enhancing the initial dataset with the goal of making it open-source. This process will adhere to domain best practices and relevant regulations.
- **Evaluation of Existing Solutions:** Studying and selecting existing tools such as language models, prompting strategies, retrieval-augmented generation (RAG), and LLM agents to serve as foundational components for the agent’s design.
- **Methodological Research:** Conducting research on reliability and robustness, with a focus on qualifying the validity and legitimacy of information provided by the agent, particularly regarding ESG (Environmental, Social, and Governance) compliance.
- **Agent Development:** Designing and developing the agent based on the selected methodologies and tools.
- **Evaluation and Deliverables:** Delivering a complete project, including the implementation of a GitHub repository with packaged algorithms shared under an open-source license.

The following diagram provides an overview of the project’s key functionalities and expected outcomes.

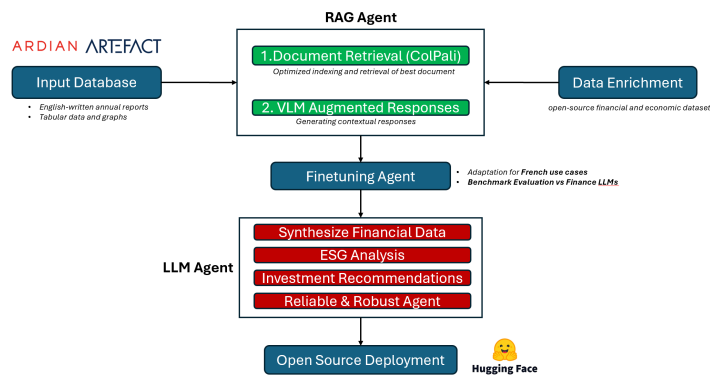


Figure 1: Project Diagram

This project aims to build a reliable and interpretable agent, advancing the state of generative AI while ensuring compliance with ESG standards and promoting collaboration in the AI community.

2 Project Organization

This section provides an overview of the project organization, including task planning, execution methodology, and work distribution among team members.

2.1 Task Planning and Monitoring

The project was structured into distinct phases, each with clear deliverables and timelines. The following Gantt chart illustrates the planning and progression of different milestones and work packages throughout the development cycle:

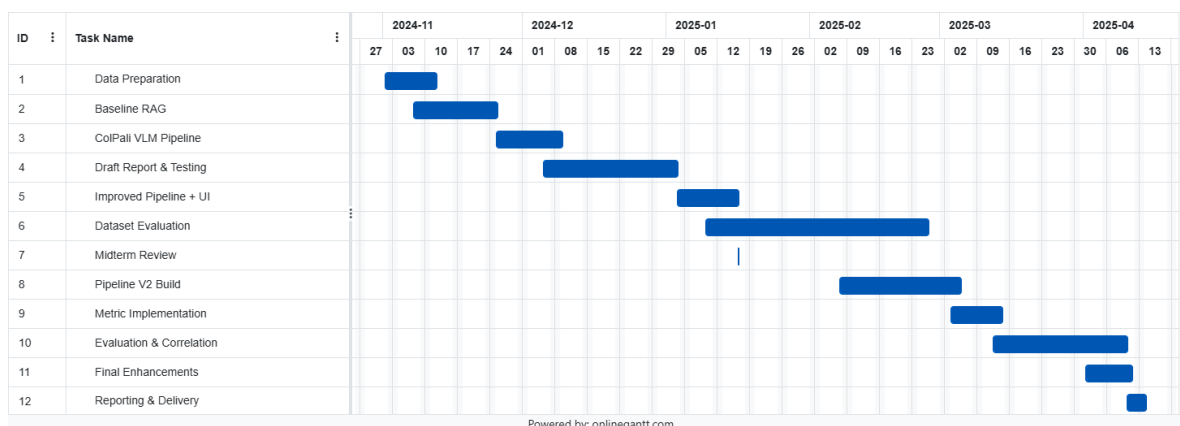


Figure 2: Gantt Chart – Project Organization

2.2 Tasks carried out

The following figure summarizes the tasks completed during this project:

N° Task ID	Aa Task name	Status	Assignee	Due	Priority
AMH-15	Prise en main RAG langchain	Done	Gabriel Trier R Rayane Bouaita L Lucas Tramonte	7 de novembro de 2024	High
AMH-14	Recherche data set open source	Done	Gabriel Trier R Rayane Bouaita L Lucas Tramonte	7 de novembro de 2024	Medium
AMH-11	Appréhender le jeu de données	Done	Lucas Tramonte R Rayane Bouaita G Gabriel Trier	11 de novembro de 2024	High
AMH-8	Statistique Dataset	Done	R Rayane Bouaita L Lucas Tramonte G Gabriel Trier	14 de novembro de 2024	High
AMH-10	User story - Cas d'usage	Done	R Rayane Bouaita L Lucas Tramonte G Gabriel Trier	21 de novembro de 2024	Medium
AMH-12	Documentation RAG Vision	Done	R Rayane Bouaita L Lucas Tramonte G Gabriel Trier	22 de novembro de 2024	Medium
AMH-22	Statistique Zipf law	Done	R Rayane Bouaita L Lucas Tramonte G Gabriel Trier	3 de dezembro de 2024	
AMH-16	Accès aux ressources GPU	Done		13 de dezembro de 2024	High
AMH-9	Adapter l'approche RAG avec Colpali	Done	G Gabriel Trier R Rayane Bouaita L Lucas Tramonte		High
AMH-13	Schéma fonctionnel - Pipeline RAG	Done	R Rayane Bouaita L Lucas Tramonte G Gabriel Trier	18 de dezembro de 2024	Medium
AMH-27	Clean Code	Done			
AMH-24	Préprocessing Fichier PDFs	Done	R Rayane Bouaita		Medium
AMH-25	Rapport du projet	Done	G Gabriel Trier L Lucas Tramonte R Rayane Bouaita	20 de dezembro de 2024	High
AMH-26	Ajout de Qwen VL pour generation	Done	G Gabriel Trier	3 de janeiro de 2025	Medium
AMH-30	Tester colpali + Qwen sur dataset annoté (réponse attendu)	Done	L Lucas Tramonte		High
AMH-31	Amélioration Pipeline génération	Done	G Gabriel Trier R Rayane Bouaita		Medium
AMH-21	Recherche: Comment tester les performances?	Done	R Rayane Bouaita L Lucas Tramonte G Gabriel Trier		High
AMH-28	Evaluer les performances (code)	Done	L Lucas Tramonte G Gabriel Trier		High
AMH-32	Faire interface web pour déposer fichier, visualiser	Done	L Lucas Tramonte G Gabriel Trier		Low

Figure 3: Tasks Completed

2.3 Collaborative Tools

- Communication: Microsoft Teams, Notion
- Document and Report Sharing: Overleaf | Latex
- Code Repository:  GitHub - Private, access available upon request.

3 Data Overview

The data used in this analysis comes from the company [Ardian](#). The dataset consists of 107 annual reports, all of which are written in English and include a dedicated section on financial analysis. A [statistical analysis](#) was conducted on the data, revealing the following insights:

On average, each document is 78 pages long, which suggests that chunking the data by page could be an effective approach. Additionally, each document contains both tabular data and graphs, which should be made exploitable for further analysis. This highlights the need for a Retrieval-Augmented Generation (RAG) Vision model, and therefore, a Vision-Language Model (VLM) could be essential for this task.

The reports provide several types of exploitable information, including financial performance indicators, strategic plans, and growth initiatives. They also contain insights into sector trends and market conditions, as well as information on ESG (Environmental, Social, and Governance) commitments. For instance, the reports highlight the carbon neutrality efforts of companies like *Emerson* and *Equity Trustees*. Finally, the documents cover risk management and governance practices, which are critical for understanding the companies' operations and strategic decisions.

3.1 User stories

We defined several user stories to guide the practical use cases of the agent and ensure it meets real-world needs. These user stories focus on key financial operations and decision-making processes.

3.1.1 Financial Summaries

- **Need:** The goal is to obtain concise financial summaries for internal follow-ups, corporate culture, and investor meetings in order to communicate company performance effectively.
- **LLM Agent:** The LLM will extract and synthesize key financial data, such as revenue, EBITA, margins, and provide a clear and concise summary.
- **Database:** The LLM will leverage the financial analysis section, which is presented in both tabular format and as general textual data.

3.1.2 ESG Practices Analysis

- **Need:** To obtain a detailed evaluation of environmental, social, and governance (ESG) practices of a company in order to align with its values.
- **LLM Agent:** The LLM will analyze ESG reports of companies in order to extract ESG initiatives (e.g., carbon emission reduction goals, diversity policies) and provide an assessment of their compliance with ESG standards.
- **Database:** The LLM will utilize the section on sustainable initiatives, carbon reduction goals (e.g., EMR_2022 document), energy transition challenges, and environmental strategies (e.g., ASX_88E document).

3.1.3 Investment & Portfolio Diversification

- **Need:** To receive recommendations for diversifying a client’s investments (or one’s own) while minimizing risks, integrating high-growth sectors.
- **LLM Agent:** The LLM will analyze geographical revenue distribution and product diversification (e.g., AMEX_EMR), as well as strategic acquisitions and strategies (e.g., ASX_EQT).

4 State of the Art

4.1 LLMs Finance

With the rise of language models in specialized fields, finance has emerged as a key sector where these technologies can transform practices. Several models have been developed to address the specific needs of this industry, combining advanced architectures and high-quality financial data. Among them, BloombergGPT and FinBERT stand out for their approaches and capabilities.

BloombergGPT is a proprietary model developed by Bloomberg, designed to meet the specific requirements of the financial sector. This model is based on a Transformer architecture similar to GPT and has been fine-tuned on an extensive corpus of financial data. These data, sourced from exclusive materials such as market reports, regulatory documents, and financial news articles, enable the model to grasp the nuances of financial language and concepts. BloombergGPT is used to generate financial reports, analyze complex regulatory texts, and automate the interpretation of

market trends. However, its access remains limited as it is proprietary, which restricts its usage outside of Bloomberg-developed solutions.

FinBERT, on the other hand, is an open-source model based on BERT, specifically adapted for financial tasks. Trained on accessible data such as corporate reports and economic articles, it excels at tasks like sentiment classification—essential for determining whether a financial text is positive, negative, or neutral—and the extraction of entities specific to the domain. FinBERT is particularly valued for its effectiveness in targeted analyses, though it is less suited for text generation compared to models like BloombergGPT.

To evaluate the coherence and accuracy of the responses generated by the model we are developing, one of these specialized models, will be used as a reference to compare results and ensure a high level of quality and relevance.

4.2 Domain Specialization

Ling et al. (2023) argue that domain specialization plays a role in enhancing the capabilities of Large Language Models (LLMs), enabling them to deliver disruptive outcomes across various industries. While general-purpose LLMs, such as GPT-based models, exhibit remarkable versatility, their performance in domain-specific tasks is often suboptimal due to the lack of fine-tuning on the specialized vocabulary, domain knowledge, and contextual understanding unique to particular fields. The authors advocate for the use of domain-adapted models, which are trained or fine-tuned on datasets that specifically reflect the terminology, structures, and nuances of the target domain. Such specialization allows the model to achieve greater accuracy, reliability, and interpretability when tasked with domain-specific queries [8].

In the context of financial analysis and ESG reporting, as outlined in the current project, domain specialization is particularly pertinent. Ling et al. emphasize the **inadequacies of general LLMs** in dealing with highly structured, domain-specific data, such as financial indicators (e.g., EBITDA, revenue growth, risk ratios) and ESG metrics (e.g., carbon emission reductions, diversity metrics, governance structures). These models are often ill-equipped to accurately interpret the contextual significance of such indicators without prior exposure to domain-specific knowledge. The authors suggest that specialized training data, incorporating both **quantitative data** (e.g., financial tables) and **qualitative data** (e.g., narrative reports), allows LLMs to perform better in understanding the intricate relationships between the different elements in financial documents and sustainability reports.

This research directly informed the approach taken in the project. To enhance the LLM’s ability to analyze financial and ESG reports, the system was designed to incorporate domain-specific fine-tuning. The focus was on adapting the model to better interpret sector-specific metrics, sustainability goals, and governance structures typically found in financial documents. Following the methodology described by Ling et al., the LLM agent was fine-tuned on a carefully curated dataset of financial reports and ESG disclosures, improving its ability to identify and synthesize the most relevant insights, especially within the context of specific companies or industries.

Moreover, Ling et al. emphasize the importance of incorporating domain-specific data not only during model training but also in the **retrieval and augmentation stages**. The combination of a specialized retrieval-augmented generation (RAG) architecture with a Vision-Language Model (VLM) is in line with their findings that such hybrid models are better suited for tasks requiring a deep understanding of both textual and visual data. In this project, the integration of ColPali

for document retrieval, coupled with Qwen-VL for response generation, **aligns with Ling et al.’s recommendation to combine advanced retrieval techniques with domain-adapted LLMs** to achieve superior results in specialized tasks.

By leveraging domain-specialized models and incorporating relevant datasets, this approach aims to significantly improve the model’s accuracy in extracting and interpreting critical financial and ESG data. This directly aligns with Ling et al.’s assertion that domain adaptation is not only necessary for improving the interpretability and performance of LLMs but is also pivotal for ensuring the reliability and robustness of the insights generated by such models in high-stakes domains like finance and sustainability.

4.3 Data Enrichment

In order to enrich the existing dataset, several open-source financial and economic datasets were explored. Initially, two datasets were chosen as benchmarks due to their relevance and accessibility:

1. [Finance Alpaca](#) provides a collection of financial market data, including quotes, historical prices, and various financial indicators for assets. The dataset has been cleaned and is available for use in its processed form at [Wealth Alpaca Dataset](#). Additionally, a GitHub repository with performance analysis, training scripts, data generation scripts, and inference notebooks can be found [here](#).

2. [Financial Reports SEC Dataset](#) provides free access to the financial reports of publicly listed companies in the United States, as filed with the Securities and Exchange Commission (SEC). It includes balance sheets, income statements, and other regulatory filings.

In addition to these benchmark datasets, the [OSCAR Dataset](#) was considered for the translation of documents into languages other than English. OSCAR is a multilingual dataset sourced from web content and is widely used for natural language processing (NLP), machine translation, and AI research. This dataset can be accessed [here](#).

Furthermore, several other datasets, while less central to the project, could serve as additional sources of enrichment for the dataset. These include:

- [ECB SDW Dataset](#) (European Central Bank Statistical Data Warehouse) offers extensive statistical data on the economic and monetary policies of the European Central Bank (ECB). It includes detailed data on inflation, interest rates, monetary aggregates, and other economic indicators from the Eurozone and its member states.

- [OECD Economic Outlook Dataset](#) provides economic forecasts and analyses of global trends for OECD member countries. It includes macroeconomic indicators, growth projections, unemployment rates, and reports on international economic policies.

- [INSEE](#) (Institut National de la Statistique et des Études Économiques) publishes official statistical data for France, covering a wide range of sectors such as the economy, population, employment, and more. The dataset includes national and regional economic indicators, as well as census data.

- [CNet Dataset](#) is a dataset of web text data gathered from Common Crawl, a project that collects data from the web for NLP and machine learning applications. This dataset is available for download, although it often requires preprocessing to be useful in specific applications.

By combining these datasets, the dataset’s scope and diversity can be significantly expanded, facilitating more robust analysis and offering a broader set of insights into financial performance, market trends, and other related topics.

5 Technical Pipeline

5.1 Getting Started with LangChain

5.1.1 Retrieval-Augmented Generation (RAG)

One of our initial objectives was to familiarize ourselves with commonly used tools to build a custom Retrieval Augmented Generation (RAG) pipeline and test it with the project’s dataset.

A standard RAG pipeline is composed of two primary parts:

1. **Indexing:** This step involves preprocessing the dataset by splitting documents into manageable chunks, generating vector embeddings for these chunks, and storing them in a vector database for efficient retrieval.
2. **Retrieval:** At query time, relevant chunks are retrieved from the vector database based on semantic similarity to the input query. These chunks are then passed to a language model for response generation.

Figure 4 below illustrates a typical RAG pipeline, highlighting the main components and their interactions.

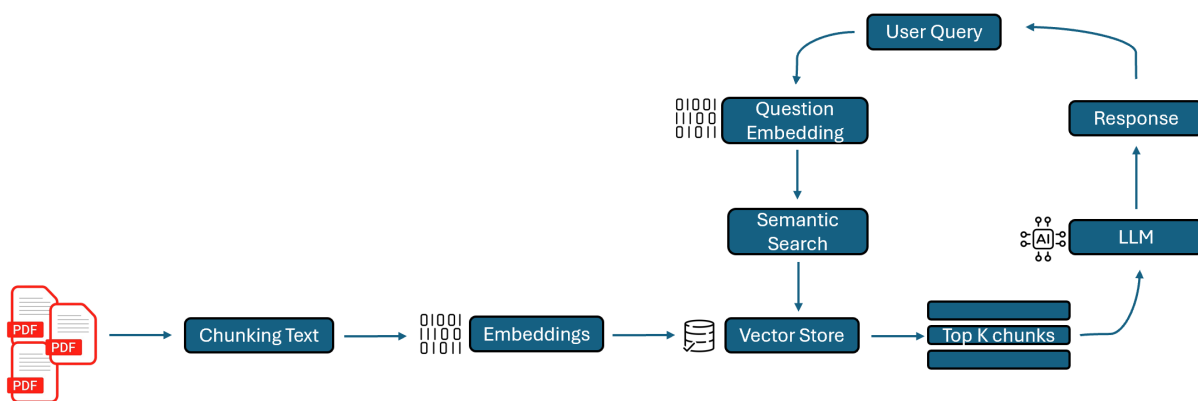


Figure 4: Schema of a classic RAG Pipeline.

5.1.2 First LangChain Integration

LangChain is a modular framework designed to simplify the development of AI-driven pipelines by integrating a variety of tools.

In our experimental pipeline, we leveraged LangChain’s flexibility to process the document corpus of our dataset and generate responses to specific queries. We used the following key elements provided by LangChain:

- **Document Loaders:** Tools to load and extract content from different document formats. In our case, we extracted data from PDF files.
- **Text Splitters:** Documents were split into smaller chunks to enable efficient processing.
- **Embeddings:** Convert text chunks into numerical vectors to enable semantic search.
- **Vector Stores:** Embeddings were stored in a FAISS (Facebook AI Similarity Search), an

open-source library designed for fast similarity search in high-dimensional spaces, index for rapid and efficient retrieval.

- **Retrievers:** Fetch relevant document chunks based on the input query.
- **LLMs (Language Models):** We used the Flan-T5 model for query interpretation and response generation, as it effectively handles French-language queries. Due to hardware limitations, we couldn't use larger models like Mistral for this preliminary exploration.
- **Chains:** Orchestrate the workflow, managing the flow of information between components.

The figure below (Figure 5) illustrates an example output of our pipeline when tested on a specific query.

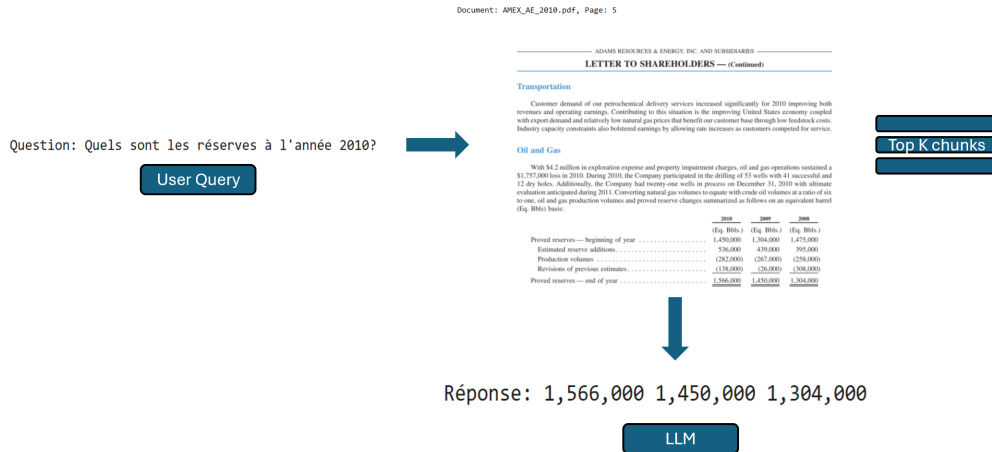


Figure 5: Traditional RAG Vision Pipeline

We can observe that the pipeline successfully retrieves the correct document and locates the page containing the relevant information. However, it struggles processing poorly structured tabular and textual data, which are key limitations. These limitations restrict its applicability in scenarios requiring a multimodal understanding of the data.

5.1.3 Improved LangChain version

In this improved version of the LangChain pipeline, we rethought several key steps to overcome the limitations of the initial approach and optimize text processing. The goal was twofold: enhance the quality of information retrieval and leverage a more advanced instruction-tuned model to generate precise and instructive answers.

Overall Pipeline Architecture

This new pipeline is divided into two main stages:

1. Document Retrieval with LangChain:

- **Document Loading and Splitting:** We use `PyMuPDFLoader` to extract content from PDF files, followed by `RecursiveCharacterTextSplitter`, a LangChain utility for split-

ting text into semantically coherent chunks, to break the text into optimally sized segments. This segmentation ensures that each chunk maintains a coherent context, facilitating more effective vectorization.

- **Embedding Generation:** Each text chunk is converted into a numerical vector using `HuggingFaceEmbeddings`, with the model `all-MiniLM-L12-v2`, which is a lighter and faster transformer model designed for generating dense vector embeddings from text. This model is particularly effective for semantic search as it can represent textual information in high-dimensional space (up to 768 dimensions). This step captures the semantic meaning of the text, enabling a refined semantic search.
- **Indexing with FAISS:** The obtained vectors are stored in a FAISS index, providing rapid semantic search capabilities. At query time, this index retrieves the most similar text chunks based on their semantic relevance.

2. Response Generation with Qwen2.5-VL-7B-Instruct:

- **Prompt Construction:** The retrieved chunks are integrated into a structured prompt that combines the extracted information with the user's query. This enriched prompt ensures that the model receives all necessary context.
- **Final Answer Generation:** The complete prompt is fed to the **Qwen2.5-VL-7B-Instruct** model, which, thanks to its advanced comprehension and generation capabilities, produces a final answer that is precise and instructive, well-suited to the complexity of the query.

Enhancements and Optimizations Over the Initial Version

Compared to the initial LangChain integration using the **Flan-T5** model, this improved version offers several enhancements:

- **Optimized Document Handling:** The use of `RecursiveCharacterTextSplitter` allows for a finer and contextually coherent segmentation of documents. This minimizes information loss during splitting and enhances retrieval quality.
- **Enhanced Semantic Search:** The combination of `HuggingFaceEmbeddings` (using the `all-MiniLM-L12-v2` model) with FAISS indexing results in more relevant matches during retrieval, based on a deeper semantic understanding of the content.
- **Reduced Computational Complexity:** By focusing on a purely text-based approach, the pipeline avoids the additional processing overhead related to multimodal models, thus enabling faster prototyping and reducing hardware demands.
- **Improved Response Generation:** Integrating the **Qwen2.5-VL-7B-Instruct** model allows the system to better handle complex queries by generating high-quality, contextually enriched responses that significantly outperform the previous setup.

Benefits and Outcomes

This enhanced pipeline maintains the core advantages of the traditional RAG approach while delivering noticeable improvements:

- Better precision in document retrieval, thanks to advanced text segmentation and vectorization.
- A simplified text-based pipeline that reduces hardware requirements and accelerates the de-

velopment process.

- Superior capability to generate high-quality, contextually relevant responses, facilitated by a state-of-the-art instruction-tuned model.

In conclusion, the integration of **Qwen2.5-VL-7B-Instruct** into the LangChain pipeline represents a significant evolution from the initial implementation. It not only optimizes the indexing and retrieval processes but also significantly enhances the quality of the final responses.

However, despite these improvements, the results are not entirely satisfactory. The current system struggles with interpreting the visual aspects of documents, which can result in incomplete analyses and occasionally less satisfying answers. These limitations highlight the need for further exploration by incorporating visual processing capabilities.

5.2 RAG Vision approach

To overcome these challenges from our LangChain approach, we explored Retrieval-Augmented Generation (RAG) Vision as an alternative. This approach combines information retrieval and generative models to deliver precise and contextually enriched responses. RAG Vision systems retrieve relevant documents or visual data and use them to answer questions, generate summaries, or provide context-aware insights. This makes them highly effective for domains like finance, healthcare, legal analysis, and education, where handling both structured and unstructured data is essential.

5.2.1 Challenges in RAG vision

Despite their potential, nowadays RAG vision systems face significant limitations when dealing with visually rich documents. These challenges include:

- **Text-Centric Approaches:** Traditional pipelines heavily rely on Optical Character Recognition (OCR) to extract text, often neglecting the rich visual context embedded in charts, layouts, and infographics.
- **Inefficient Preprocessing:** Separate steps like OCR, layout detection, and chunking introduce latency and increase the computational cost, limiting scalability for large datasets.
- **Limited Multi-Modal Integration:** Existing systems struggle to fully integrate visual and textual features, reducing their effectiveness for tasks requiring a holistic understanding of document content.

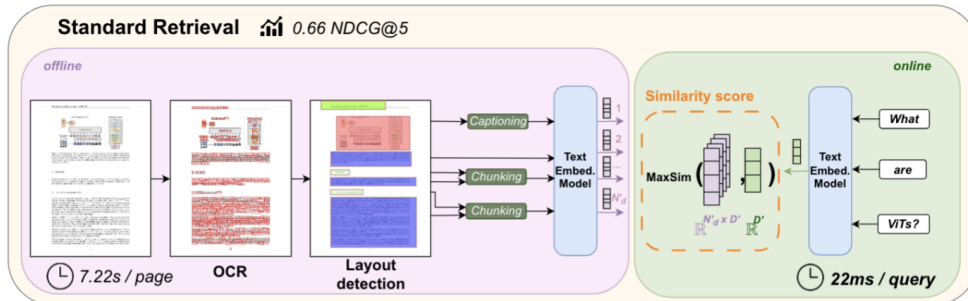


Figure 6: Tradition RAG Vision Pipeline

5.2.2 ColPali

ColPali is an innovative retrieval system designed to overcome the challenges of visually rich document retrieval. Unlike traditional pipelines that rely heavily on OCR and text-based indexing, ColPali leverages the latest advancements in Vision Language Models (VLMs) to directly embed document page images and perform efficient query matching using late interaction mechanisms.[3]

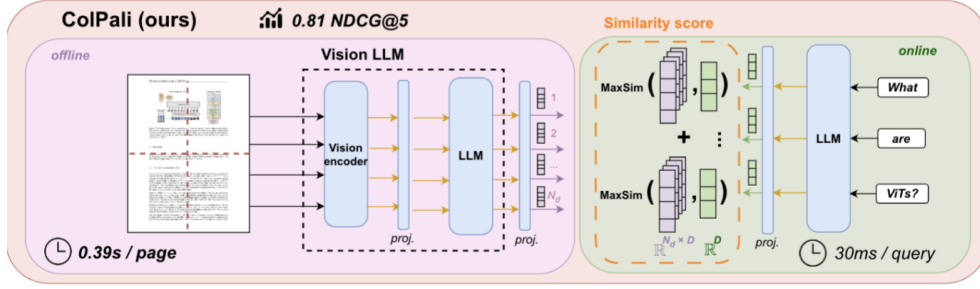


Figure 7: Colpali Pipeline

5.2.3 Indexing Phase

The indexing phase in ColPali aims to simplify traditional document ingestion pipelines by processing each page of the document as an image directly.

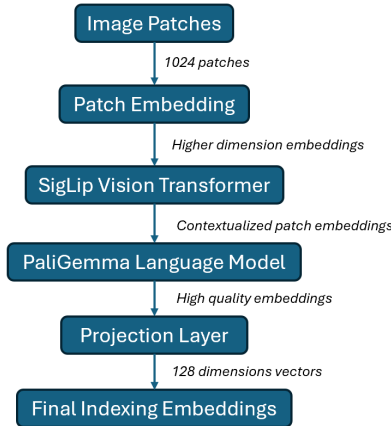


Figure 8: Schema of indexing Colpali pipeline

Below is a detailed breakdown of each step in the indexing phase:

1. Image Patch Extraction The first phase consists in dividing each image into a grid of smaller image patches. This process ensures that both local and global visual information, such as spatial structures, layouts, and visual elements like charts or tables, is captured.

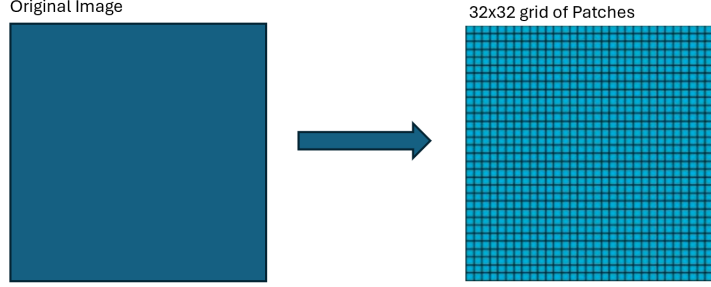


Figure 9: Schema of image patching process

2. Patch Embedding The individual image patches are then embedded into high-dimensional vectors, to represent their visual features.

- A linear projection layer is applied to each image patch to transform the raw pixel data into meaningful patch embeddings.
- This step ensures that each patch is represented in a way that highlights its unique visual characteristics.

3. Vision Transformer (SigLIP-So400m) The resulting patch embeddings are processed by a Vision Transformer (ViT), the SigLIP-So400m model, to capture complex relationships between the patches. [4]

- The Vision Transformer models the interaction between patches using self-attention mechanisms.
- It encodes spatial and structural relationships between different regions of the document.

The output of this process is a contextualized patch embeddings, which incorporate both local and global context within the document. This step ensures that the relationships between the patches are fully understood.

4. Language Model (Gemma-2B) The contextualized patch embeddings are inputted as “soft tokens” into the PaliGemma-2B language model, a Vision-Language Model (VLM), to align the visual features with semantic textual representations.

The language model processes the contextualized patch embeddings as if they were language tokens, producing high-quality embeddings that integrate both visual and semantic context.

5. Final Patch Embeddings Finally, the embeddings from the language model are projected into a lower-dimensional latent space ($D = 128$) for efficient storage and faster retrieval. This lower dimensional output is a multi-vector representation of the document page, where each vector corresponds to a specific patch.

$$\min (\text{distance_totale} + \alpha \cdot \text{varMaxLoad} + \beta \cdot \text{varNbRelocVar})$$

où distance_totale est la somme des distances entre la brique bureau et les briques assignées.

5.2.4 Retrieval Phase

The retrieval phase in ColPali efficiently matches user queries to relevant document pages using a late interaction mechanism. This phase focuses on leveraging the pre-indexed embeddings for fast and precise query matching.

1. Query Embedding At runtime, the user query is embedded by the Gemma-2B Language Model into a series of token embeddings, each representing a specific term in the query.

2. Late Interaction Mechanism ColPali employs a ColBERT-style late interaction (LI) approach to match query tokens with document patches. For each term in the query, the system identifies the document patch that has the most similar representation based on the ColPali embeddings. The late interaction mechanism ensures that every term in the query interacts with all document patches. It combines the strengths of bi-encoder models (offline computation and fast retrieval) with the detailed matching capabilities of cross-encoders. [6] [4]

Late Interaction. Given a query q and a document d , let $\mathbf{E}_q \in \mathbb{R}^{N_q \times D}$ and $\mathbf{E}_d \in \mathbb{R}^{N_d \times D}$ represent their respective multi-vector embeddings in the shared embedding space $\mathbb{R}^D$. The late interaction operator $LI(q, d)$ is computed as:

$$LI(q, d) = \sum_{i=1}^{N_q} \max_{j=1}^{N_d} \langle \mathbf{E}_q^{(i)}, \mathbf{E}_d^{(j)} \rangle$$

With:

- $\mathbf{E}_q^{(i)}$ is the i -th query embedding vector.
- $\mathbf{E}_d^{(j)}$ is the j -th document embedding vector.
- $\langle \cdot, \cdot \rangle$ denotes the dot product.

Here is an illustration of the late interaction process : [6]

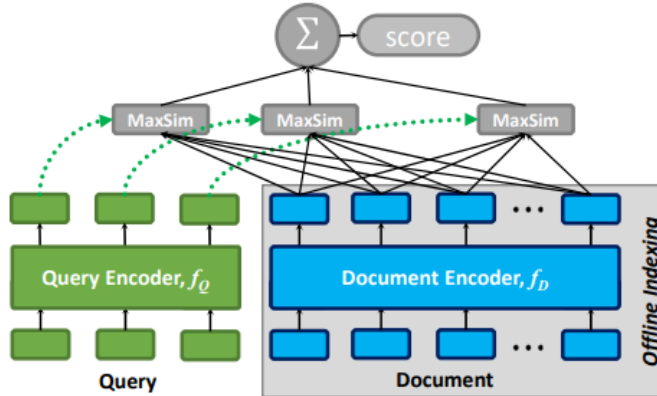


Figure 10: Illustration of Late Interaction Mechanism in ColPali

3. Query to Document Scoring For each query token, the document patch with the highest similarity score is selected. The final query-to-document score is computed by summing these

maximum scores across all query tokens. This ensures a comprehensive interaction between the query and document, incorporating both semantic and visual context.

5.2.5 Performances

ColPali is benchmarked on the ViDoRe dataset, which evaluates retrieval systems on tasks involving visually rich documents across domains such as healthcare, finance, and academia. ViDoRe tasks include retrieving tables (TabFQuAD), scientific figures (ArxivQA), and infographics (InfoVQA), providing a comprehensive test of ColPali’s capabilities.

Dataset	# Queries	Domain
Academic Tasks		
DocVQA (eng)	500 (500)	Industrial
InfoVQA (eng)	500 (500)	Infographics
TAT-DQA (eng)	1600 (1600)	Varied Modalities
arXivQA (eng)	500 (500)	Scientific Figures
TabFQuAD (fra)	210 (210)	Tables
Practical Tasks		
Energy (eng)	100 (1000)	Scientific
Government (eng)	100 (1000)	Administrative
Healthcare (eng)	100 (1000)	Medical
AI (eng)	100 (1000)	Scientific
Shift Project (fra)	100 (1000)	Environment

Figure 11: Composition of the ViDoRe benchmark.

ColPali demonstrates state-of-the-art performance on the ViDoRe benchmark, significantly outperforming all baseline models. [4]

- **High Retrieval Performance:** ColPali outperforms existing models on diverse tasks such as tables (TabFQuAD), infographics (InfoVQA), and scientific figures (ArxivQA), with an average NDCG@5 score of 81.3, compared to 67.0 for the closest baseline.

	ArxivQ	DocQ	InfoQ	TabF	TATQ	Shift	AI	Energy	Gov.	Health.	Avg.
Unstructured - test only											
- BM25	-	34.1	-	-	44.0	59.6	90.4	78.3	78.8	82.6	-
- BGE-M3	-	28.4 _{±5.7}	-	-	36.1 _{±19.9}	68.5 _{±8.9}	88.4 _{±20.9}	76.8 _{±13.3}	77.7 _{±11.1}	84.6 _{±10.0}	-
Unstructured - ocr											
- BM25	31.6	36.8	62.9	46.5	62.7	64.3	92.8	85.9	83.9	87.2	65.5
- BGE-M3	31.4 _{±0.2}	25.7 _{±11.3}	60.1 _{±28.8}	70.8 _{±24.3}	50.5 _{±12.2}	73.2_{±8.9}	90.2 _{±26.6}	83.6 _{±23.3}	84.9 _{±10.0}	91.1 _{±19.9}	66.1 _{±0.6}
Unstructured - Captioning											
- BM25	40.1	38.4	70.0	35.4	61.5	60.9	88.0	84.7	82.7	89.2	65.1
- BGE-M3	35.7 _{±4.4}	32.9 _{±8.4}	71.9 _{±11.9}	69.1 _{±19.7}	43.8 _{±10.7}	73.1 _{±12.2}	88.8 _{±0.8}	83.3 _{±14.4}	80.4 _{±23.3}	91.3 _{±21.1}	67.0 _{±1.9}
Contrastive VLMs											
Jina-CLIP	25.4	11.9	35.5	20.2	3.3	3.8	15.2	19.7	21.4	20.8	17.7
Nomic-vision	17.1	10.7	30.1	16.3	2.7	1.1	12.9	10.9	11.4	15.7	12.9
SigLIP (vsnlto)	43.2	30.3	64.1	58.1	26.2	18.7	62.5	65.7	66.1	79.1	51.4
Ours											
SigLIP (vsnlto)	43.2	30.3	64.1	58.1	26.2	18.7	62.5	65.7	66.1	79.1	51.4
BiSigLIP (vsnlto+img)	58.5 _{±15.3}	32.9 _{±28.8}	70.5 _{±6.4}	62.7 _{±4.6}	30.5 _{±13.3}	26.5 _{±7.8}	74.3 _{±11.8}	73.7 _{±8.9}	74.2 _{±8.1}	82.3 _{±12.2}	58.6 _{±1.2}
BiPali (vsnlto)	56.5 _{±20.9}	30.0 _{±29.8}	67.4 _{±31.1}	76.9 _{±14.2}	33.4 _{±19.9}	43.7 _{±17.2}	71.2 _{±3.1}	61.9 _{±10.7}	73.8 _{±0.4}	73.6 _{±4.8}	58.8 _{±0.2}
ColPali (vsnlto+img)	79.1_{±22.6}	54.4_{±24.3}	81.8_{±10.8}	83.9_{±7.0}	65.8_{±12.4}	73.2_{±28.5}	96.2_{±30.9}	91.0_{±20.1}	92.7_{±0.0}	94.4_{±0.8}	81.3_{±22.3}

Figure 12: Results on Vidore Benchmark

- **Faster Indexing:** By bypassing the need for complex preprocessing pipelines like OCR and layout detection, ColPali reduces indexing latency by a factor of 18, compared to standard retrieval systems.

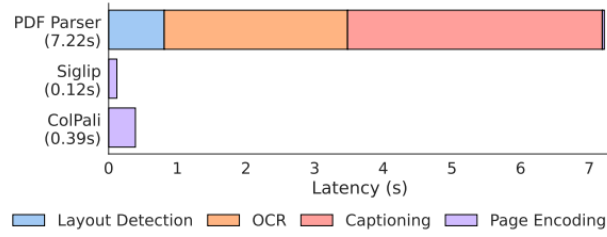
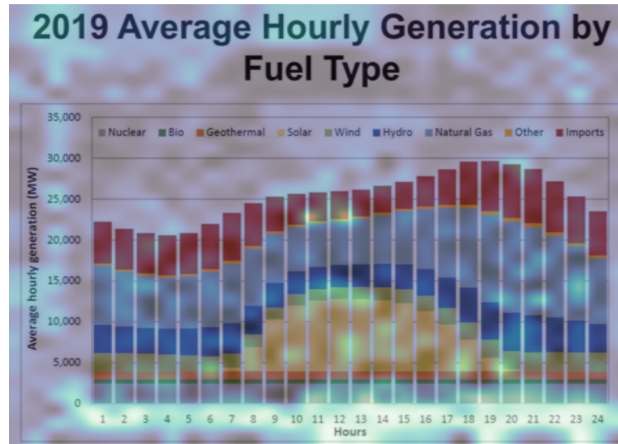


Figure 13: Offline indexing performances

5.2.6 Conclusion

To conclude, ColPali stands out as a powerful tool for visually rich document retrieval, particularly in our project context, where interpretability and explainability are key concerns.



Query: "Which hour of the day had the highest overall eletricity generation in 2019?"

Figure 14: Interpretability of the most relevant document image patches for query.

Its ability to generate visual heatmaps, as shown in Figure 14, allows us to identify which document patches are most relevant to a given query. [3]

Beyond interpretability, ColPali is well-suited for our goals:

- **Performance in Specific Domains:** It excels in tasks involving finance and other highly structured fields, aligning with the project’s requirements.
- **Modular Integration:** We can leverage ColPali for the indexing phase, efficiently creating contextualized embeddings from visually rich documents. For the retrieval phase, a complementary LLM or VLM can provide additional context and enhance the quality of responses.

However, certain challenges must be addressed:

- **Limited Vector Store Support:** Current vector databases lack native support for multi-vector embeddings, requiring on-disk storage solutions.

- **Training Limitations:** ColPali is trained predominantly in English, which might lead to performance drops in French domain-specific applications.

6 RAG vision Pipeline Description

Our final pipeline consists of two main stages, document retrieval and VLM augmented answers.

In the first part, we leverage the Colpali model to retrieve the most relevant documents corresponding to the user query.

In the second part, we use Qwen-VL, a powerful Vision-Language Model (VLM) to generate accurate and contextualized answers by combining the retrieved documents and the user query within a structured prompt.

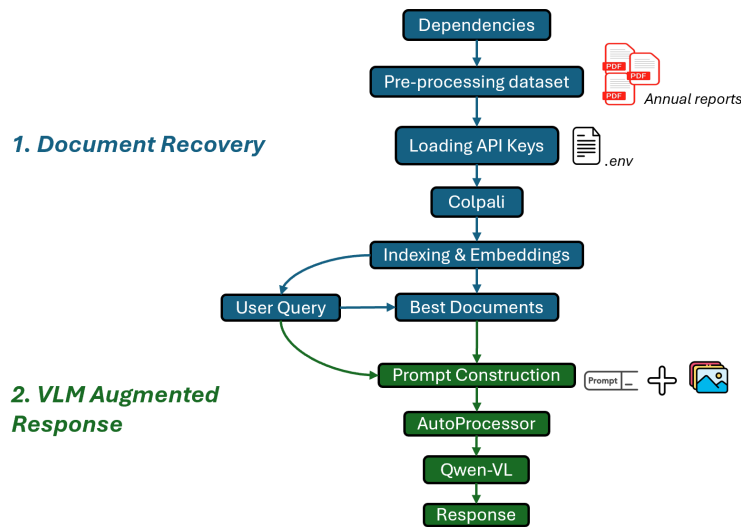


Figure 15: Overview of the Final Pipeline Architecture

6.1 Pipeline Steps

The pipeline is divided into the following steps:

- **Dependencies and Initialization:** We load the necessary libraries, ColPALI model for document retrieval, Qwen-VL for response generation, and API keys.
- **Document Indexing:** Using ColPALI, the input PDFs are pre-processed and indexed. This step prepares the documents for fast retrieval based on query similarity.
- **Query-Based Retrieval:** Given a user query, the most relevant documents are identified using cosine similarity between query embeddings and document embeddings.
- **Interpretability with Similarity Maps:** For the top- k retrieved documents, we generate token-based similarity maps to highlight the regions most relevant to the query. These maps provide a visual representation of how the model interprets the input.
- **Response Generation:** The user query and the most relevant documents are combined into a structured prompt. This prompt is passed to our generation VLM model, which generates

a contextualized and accurate response.

7 Generative Models for Response Formulation

Once the retrieval component has identified the most relevant document pages, the final stage of the pipeline involves generating a coherent and accurate answer based on the user’s query and the retrieved context. This step is handled by a Large Language Model (LLM) or a Vision-Language Model (VLM), depending on the pipeline configuration. To evaluate the effectiveness of different generative capabilities, we selected four distinct models, varying in size, architecture, and modality (text-only vs. vision-language).

The models chosen for the generation task are:

- **Google Gemma 3 (4B and 12B):** Representing state-of-the-art text-based LLMs.
- **Qwen2 VL (2B and 7B):** Representing powerful Vision-Language Models capable of interpreting both text and images.

7.1 Google Gemma 3 Models

The Gemma models, developed by Google DeepMind, are a family of lightweight, open-weight language models built from the same research and technology used to create the Gemini models. We specifically evaluated two instruction-tuned variants from the recent Gemma 3 release:

- **google/gemma-3-4b-it:** A 4 billion parameter model optimized for instruction following. It offers a balance between performance and computational efficiency.
- **google/gemma-3-12b-it:** A larger 12 billion parameter instruction-tuned model from the same family. This model provides enhanced reasoning and generation capabilities compared to its smaller version.

Both Gemma 3 models are primarily text-based, designed to process and generate natural language based on the textual context provided (query + retrieved text snippets). Their instruction tuning makes them adept at following specific commands and formatting requirements within the prompt.

7.2 Qwen2 VL Models

The Qwen models are developed by Alibaba Cloud’s Qwen team. The Qwen2 series represents their second generation of large-scale language models, with improvements in performance and efficiency. We focused on the Vision-Language (VL) variants, which are specifically designed to process multimodal inputs (text and images):

- **Qwen/Qwen2-VL-2B-Instruct:** A 2 billion parameter Vision-Language Model. This is a relatively compact VLM, optimized for instruction following, capable of understanding visual content from the retrieved document pages alongside the textual query.
- **Qwen/Qwen2.5-VL-7B-Instruct:** A larger 7 billion parameter VLM from a slightly updated release (Qwen 2.5). This model offers more advanced multimodal understanding and generation capabilities, benefiting from its larger size and architectural refinements.

The key advantage of the Qwen VL models in our context is their ability to directly interpret the visual information from the retrieved document pages, potentially capturing nuances lost in simple text extraction (OCR).

7.3 Model Comparison Summary

The selection of these four models allows for a comparative analysis across several dimensions:

- **Developer:** Google (Gemma) vs. Alibaba Cloud (Qwen).
- **Modality:** Text-only (Gemma) vs. Vision-Language (Qwen VL). This is a critical distinction, as VLMs can leverage the visual structure of the retrieved pages, while text models rely solely on extracted text.
- **Size (Parameters):** Ranging from 2B (Qwen2-VL) and 4B (Gemma-3) to 7B (Qwen2.5-VL) and 12B (Gemma-3). This allows evaluating the trade-offs between model capability and computational cost.
- **Recency:** All models are relatively recent releases (primarily from 2024), representing current advancements in LLM and VLM technology.
- **Tuning:** All selected models are instruction-tuned ('-it' or '-Instruct'), making them suitable for the task-oriented nature of RAG generation where following prompt instructions is essential.

By evaluating these diverse models, we aim to identify which generative approach (text-based vs. VLM) and which model size provides the best balance of accuracy, faithfulness, and efficiency for answering financial queries based on retrieved document context.

8 Pipeline V2: Enhanced Pipeline with Re-ranking and Proposed Third Iteration

8.1 Limitations of Pipeline V1

Pipeline V1 primarily relies on visual signals for document retrieval while overlooking the textual content embedded within the pages. This limitation becomes particularly evident when the input documents consist of varied formats such as result accounts, where accurate textual interpretation is needed for determining relevance. Moreover, the nature of the visual retrieval often leads to the inclusion of pages that are either blank or contain images that are not contextually appropriate. Such non-relevant pages can be returned by the retriever, introducing noise into the process. The absence of text evaluation in the initial pipeline therefore not only risks attenuating the overall relevance of the retrieved documents but also reduces the quality of the context that is later fed into the generative model. Our proposed approach aims to overcome these issues by integrating a thorough text analysis step.

8.2 Architecture and Enhancements of Pipeline V2

In Pipeline V2, we have re-designed the retrieval process to address the limitations of the initial approach by integrating textual analysis and re-ranking mechanisms. The new architecture retains the visual retrieval component based on ColPali while adding several key stages that refine the final output. In this iteration, the system is organized into a series of steps. Starting with Optical Character Recognition (OCR) to extract textual content from each page, the pipeline filters pages based on their text density. The filtered pages are then processed by the Colpali retrieval system, which returns an initial set of candidate pages (typically 15) that exhibit visual similarity to the query. Finally, a cross encoder module is applied to these candidates to re-rank them based on

semantic similarity, resulting in the selection of the top three most relevant documents. These top-ranked pages are then provided as context to the generative model, to formulate its response.

8.3 Optical Character Recognition and Preprocessing

The introduction of OCR in Pipeline V2 represents an enhancement over the previous version. Each document page is first processed through an OCR system, which extracts the textual content embedded within the images. This step is used first for evaluating the text density on each page, allowing the system to filter out pages that do not meet a minimum threshold of textual substance. By eliminating pages with insufficient text, the pipeline focuses its computational resources on content that is likely to contain valuable context, thus setting a foundation for the subsequent candidate retrieval phase.

8.4 Candidate Retrieval with Colpali

After the OCR-based filtering, the pipeline leverages the strengths of the Colpali retrieval system to quickly index and retrieve a broad set of candidate pages. This approach produces an initial candidate set of approximately 15 pages that are visually aligned with the query intent. The use of Colpali at this stage ensures that the system efficiently processes large documents by narrowing down the scope to a manageable subset of potentially relevant pages.

8.5 Detailed Analysis of the Cross Encoder Re-ranking Process

The re-ranking process is a step thought to refine the initial candidate set and ensure the retrieval of the most semantically relevant pages. Re-ranking leverages the text extracted from each page, focusing on the actual content rather than solely on the visual representation. In this stage, the re-ranking process compares the query with the text of each candidate, identifying subtle contextual cues and linguistic nuances that a visual-only approach might miss.

At the heart of this refinement stage is the cross encoder model. Unlike bi-encoder methods that encode the query and document separately, the cross encoder processes the concatenated query and document text together, facilitating joint encoding. This technique enables the model to perform token-level attention across both inputs simultaneously, thus capturing complex relationships and dependencies. The model computes a relevance score for each query-document pair by evaluating how well the combined information aligns with the query’s intent. Such a detailed assessment is coherent with the underlying purpose of re-ranking, as it utilizes deep semantic interactions that are essential for distinguishing genuinely relevant pages from those that are only superficially similar.

This integration of text-based re-ranking with the cross encoder is what ultimately distinguishes our approach. The cross encoder’s ability to finely parse and weigh each token in context reinforces the overall re-ranking strategy by ensuring that only the candidates with the highest semantic congruity are selected. The coherent collaboration between the initial visual retrieval and subsequent text-focused re-ranking minimizes the risk of false positives, ultimately leading to a more robust and reliable document retrieval process. This integrated method effectively combines the strengths of both visual and textual analysis, making it ideally suited to our complex retrieval tasks.

8.6 Recap and Future Directions

In summary, Pipeline V2 is structured as a multi-stage system where OCR-driven text filtering, visual retrieval via Colpali, and cross encoder-based re-ranking work in concert to deliver superior retrieval performance. The initial candidate generation is pruned by assessing text density, then broadened through visual analysis, and finally sharpened by the cross encoder’s semantic re-ranking. Notably, experiments with an alternative sequence that swaps the ordering of the Colpali and cross encoder modules produced similar results, which indicates that our refined hybrid approach is robust.

9 Dataset

In order to evaluate the performance of our different pipelines, we designed a comprehensive dataset. This dataset was specifically created to verify the performance of our pipelines with respect to the use cases defined earlier in the project, ensuring that our solutions meet both real-world needs and our client’s specific requirements.

9.1 Composition

The dataset consists of 50 financial questions extracted from a single PDF document and is divided into two categories based on the complexity of the expected answers:

- **Short Questions:** Focused on the extraction of concise, numerical, or factual information.
- **Long Questions:** Involving more detailed analysis and synthesis of financial, operational, or strategic information.

Each question in the dataset is enriched with extensive metadata to facilitate thorough evaluation:

- **Question ID and Text:** A unique identifier and the original question phrasing as found in the PDF.
- **Expected Answer:** A reference answer recorded for later performance evaluation, serving as the gold standard against which pipeline outputs are compared.
- **Document References:** Information about the relevant document pages (or page numbers) where the answer can be verified.
- **Subject Classification:** Questions are tagged with subject categories such as *Financial_Data*, *Investment*, *Geographic*, *Environmental*, and *Business_Strategy*. This categorization helps in assessing pipeline performance across different domains within financial reporting.
- **Generation Model Responses:** For each question, we annotate the answers produced by the different generation models we want to compare, such as `Qwen2-VL-2B-Instruct`, `Qwen2.5-VL-7B-Instruct`, `gemma-3-4b-it`, and `gemma-3-12b-it`.

9.2 Scope and Use Cases

The dataset encompasses a wide range of financial topics, ensuring that our evaluation covers both straightforward data retrieval and complex analytical tasks. Key areas include:

- **Financial Metrics:** Questions regarding net sales, adjusted EBITA margins, free cash flow, dividend per share, and pretax earnings margins serve as benchmarks for quantitative extraction.

- **Investments and M&A:** Several questions query details about strategic acquisitions (for example, those related to National Instruments or Flexim/Afag) and other investment decisions, testing the pipelines on extracting both narrative and numerical information.
- **Geographic Breakdown:** Topics such as the distribution of sales across regions (e.g., the Americas versus Asia/Middle East/Africa) are used to assess the retrieval of geographically segmented information.
- **ESG and Regulatory Issues:** Queries on environmental targets, employee diversity and engagement, and regulatory risks provide insights into how well the pipelines handle qualitative data and complex compliance information.
- **Operational and Strategic Financial Management:** The dataset also includes questions about working capital management, provisions, and future financial forecasts — areas that demand both quantitative accuracy and qualitative interpretation.

10 Evaluation

Finally, to conclude on the performance of each component of our pipelines, we separated the evaluation into two main groups: the **retriever** component and the **generator** component.

10.1 Retriever Metrics

10.1.1 Revaluation ViDoRe

To ensure our retriever performs at the state-of-the-art level, we aimed to reproduce the results reported in [3]. Our implementation of the ViDoRe/ColQwen2-v1.0 retriever achieved an average score of 89.3, confirming that our retrieval component meets established standards.

10.1.2 Retriever Metrics on Our Dataset

We evaluate the retriever using four metrics:

1. **MMR (Mean Reciprocal Rank):** Measures the effectiveness of the retriever by calculating the reciprocal of the rank of the first relevant document. Higher scores (closer to 1) indicate better performance.

$$\text{MMR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{r_i}.$$

2. **Precision at 1:** Assesses the relevance of the top retrieved document by computing the proportion of relevant documents in the first retrieved result.

$$\text{Precision@1} = \frac{R_1}{1},$$

where R_1 is 1 if the document at rank 1 is relevant, and 0 otherwise.

3. **Recall at 3:** Evaluates the ability of the retriever to retrieve all relevant documents within the top three results.

$$\text{Recall@3} = \frac{\text{Number of relevant documents in top 3}}{\text{Total number of relevant documents}}.$$

4. **Embeddings Comparison:** We use the cosine similarity, which for a query embedding q and a document embedding d is defined as:

$$\text{cosine_sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}.$$

We report both the mean and median of these similarities across queries. Additionally, we compute the Euclidean distance between embeddings:

$$\text{euclidean_dist}(q, d) = \sqrt{\sum_{j=1}^n (q_j - d_j)^2},$$

and take the mean over the dataset. Finally, a *matching score* is computed using a weighted aggregation of the normalized cosine similarity and the inverse of the Euclidean distance. For example, one formulation is:

$$\text{matching_score} = \frac{w_1 \times \text{cosine_sim}(q, d) - w_2 \times \text{euclidean_dist}(q, d)}{w_1 + w_2},$$

where w_1 and w_2 are empirically determined weights.

Table 1 shows the overall metrics across all questions.

Metrics	Colqwen base	Scaled 10 PDFs	CrossEncoder
exact_match_top1	0.3654	0.2308	0.3654
mrr	0.5929	0.4913	0.4968
precision@1	0.5192	0.4231	0.4423
precision@3	0.3269	0.2564	0.2372
precision@5	0.2308	0.2000	N/A
recall@1	0.1731	0.1410	0.1391
recall@3	0.3186	0.2497	0.2173
recall@5	0.3699	0.3202	N/A
Cosine Similarity (Mean)	0.0913	0.0566	0.0913
Cosine Similarity (Median)	0.0801	0.0527	0.0801
Euclidean Distance (Mean)	0.6953	0.6719	0.6953
Matching Score	0.1087	0.0870	0.1087

Table 1: Overall Retriever Metrics on the Dataset

For questions classified as **Long**, Table 2 presents the corresponding metrics.

Metrics	Colqwen base	Scaled 10 PDFs	CrossEncoder
exact_match_top1	0.2941	0.2059	0.2941
mrr	0.5255	0.4525	0.4525
precision@1	0.4118	0.3824	0.3824
precision@3	0.2451	0.1961	0.1961
precision@5	0.1706	0.1529	N/A
recall@1	0.1373	0.1275	0.1275
recall@3	0.2451	0.1961	0.1961
recall@5	0.2843	0.2549	N/A

Table 2: Retriever Metrics for Long-Type Questions

Similarly, for **Short**-type questions, Table 3 reports the relevant metrics.

Metrics	Colqwen base	Scaled 10 PDFs	CrossEncoder
exact_match_top1	0.4444	0.2778	0.4444
mrr	0.6870	0.5648	0.5648
precision@1	0.6111	0.5000	0.5000
precision@3	0.4630	0.3704	0.3704
precision@5	0.3444	0.2889	N/A
recall@1	0.2037	0.1667	0.1667
recall@3	0.4435	0.3509	0.3509
recall@5	0.5315	0.4435	N/A

Table 3: Retriever Metrics for Short-Type Questions

10.2 Generation Metrics

The evaluation of our generator component focuses on the quality and factuality of the generated responses. We assess this both by using the FinQA benchmark and via a custom set of metrics on our dataset.

10.2.1 FinQA Evaluation

The FinQA benchmark is designed to test Visual Language Models (VLMs) directly on their ability to generate answers from PDF inputs, without a separate retrieval step. FinQA is a large-scale dataset comprising "2.8k financial reports that yield over 8k question-answer pairs" [2]. It is designed to study numerical reasoning in financial contexts by integrating both structured evidence (such as tables and graphs) and unstructured textual information. The questions in FinQA often require multi-step reasoning—many involve arithmetic operations and logical inference based on evidence gathered from different sections of a report.

Our evaluation procedure for FinQA involves:

- Loading the dataset containing questions, reference answers, and associated PDF documents.
- Running the model directly on the PDF inputs.
- Qualitatively assessing the generated answers for coherence and contextual accuracy.

The evaluation focuses on verifying that the model produces meaningful, contextually accurate responses. Table 4 summarizes the FinQA accuracy results for the evaluated models.

Model	FinQA Accuracy
Qwen/Qwen2.5-VL-7B-Instruct	28.00%
google/gemma-3-4b-it	26.00%
OpenGVLab/InternVL2_5-8B-MPO	20.00%
Qwen/Qwen2-VL-2B-Instruct	0.00%

Table 4: FinQA Accuracy Results

10.2.2 Generator Metrics on Our Dataset

In addition to the FinQA evaluation, we evaluated our generator using several metrics that capture different aspects of the generated responses. We apply different sets of these metrics on short-answer questions versus long-answer questions, as explained below.

ROUGE Metrics (for both short and long questions) ROUGE measures the overlap between the generated text and the reference text. We employ three variants:

- **ROUGE-1:** Measures the overlap of unigrams. It is computed as:

$$\text{ROUGE-1} = \frac{\text{Number of overlapping unigrams}}{\text{Total unigrams in the reference text}}.$$

- **ROUGE-2:** Measures the overlap of bigrams:

$$\text{ROUGE-2} = \frac{\text{Number of overlapping bigrams}}{\text{Total bigrams in the reference text}}.$$

- **ROUGE-L:** Measures the length of the Longest Common Subsequence (LCS) between the generated and reference texts. One common formulation is:

$$\text{ROUGE-L} = \frac{LCS(\text{generated}, \text{reference})}{\text{length of reference text}},$$

where $LCS(\cdot)$ denotes the length of the longest common subsequence. For further details, see [7].

BERTScore (applied on long questions) BERTScore uses contextual embeddings from the model `microsoft/deberta-large-mnli` to compute the similarity between generated and reference texts. For each token, it finds the maximum cosine similarity to any token in the other text.

Let R be the set of tokens in the reference and C the set in the candidate (generated) text. Then, the precision component is:

$$P = \frac{1}{|C|} \sum_{c \in C} \max_{r \in R} \cos(\mathbf{e}_c, \mathbf{e}_r),$$

and the recall component is:

$$R = \frac{1}{|R|} \sum_{r \in R} \max_{c \in C} \cos(\mathbf{e}_r, \mathbf{e}_c).$$

The overall BERTScore is typically the harmonic mean of these two components.

We apply BERTScore only on long questions because the metric leverages rich semantic information that is most beneficial when evaluating longer, more complex responses.

Flan-T5 Perplexity (applied on long questions) Flan-T5 Perplexity is evaluated using the `google/flan-t5-large` model. Perplexity (PPL) is defined as:

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log p(x_i) \right),$$

where $\{x_1, x_2, \dots, x_N\}$ is the token sequence of the generated text and $p(x_i)$ is the model’s predicted probability for token x_i . Our evaluation uses the inverse perplexity:

$$\text{Flan-T5 Score} = \frac{1}{\text{PPL}},$$

so that higher scores indicate better alignment with fluent and contextually appropriate language.

Numerical Accuracy (applied on short questions) Numerical Accuracy is tailored for questions that involve tabular or chart data. It is defined by checking if the absolute error between the generated numerical value v_g and the reference value v_r is within a 5% tolerance:

$$\text{Numerical Accuracy} = \begin{cases} 1, & \text{if } \left| \frac{v_g - v_r}{v_r} \right| \leq 0.05, \\ 0, & \text{otherwise.} \end{cases}$$

This metric is applied on short questions because these often require precise extraction of numerical data.

Multi-modal Faithfulness (applied on both short and long questions) Multi-modal Faithfulness is a binary metric that evaluates whether the generated response contains only information present in the top 3 retrieved documents. It is defined as:

$$\text{Faithfulness} = \begin{cases} 1, & \text{if the response is fully supported by the retrieved context,} \\ 0, & \text{if it contains hallucinated information.} \end{cases}$$

String Presence (applied on short questions) The String Presence metric from RAGAS [1] checks if the expected reference text (keywords or phrases) appears in the generated response:

$$\text{StringPresence} = \begin{cases} 1, & \text{if the reference string is found in the response,} \\ 0, & \text{otherwise.} \end{cases}$$

This metric is particularly important for short answers where capturing the key phrase is critical for factual verification.

10.2.3 Aggregation of Generation Metrics

We aggregate the multiple generation metrics using the Borda count method [5]. The Borda count is a ranking-based aggregation technique that assigns points to each model based on its position in the ranking for each metric. For a given metric, if there are N models and a model is ranked r (where $r = 1$ corresponds to the best performance), then the model receives:

$$\text{Points} = N - r$$

The overall Borda score for a model is the sum of the points across all evaluated metrics. This method is particularly useful as it normalizes the different scales and units of the metrics, yielding a single aggregated score for each model.

We derive three distinct aggregated scores:

- **Global Borda Score:** Aggregates metrics across all question types.
- **Short-Answer Borda Score:** Filter short questions only and test the following metrics: ROUGE-1, ROUGE-2, ROUGE-L, String Presence, Numerical Accuracy, and Faithfulness Score.
- **Long-Answer Borda Score:** Filter long questions only and test the following: ROUGE-1, ROUGE-2, ROUGE-L, BERTScore, Flan-T5 Perplexity, and Faithfulness Score.

For each set of metrics, the Borda score of model i is computed as:

$$B_i = \sum_{j=1}^M (N - r_{ij}),$$

where M is the total number of metrics, r_{ij} is the rank of model i for metric j , and N is the total number of models.

Summing these points across all metrics yields a single *Borda score* per model. A higher Borda score indicates stronger overall performance when considering all metrics.

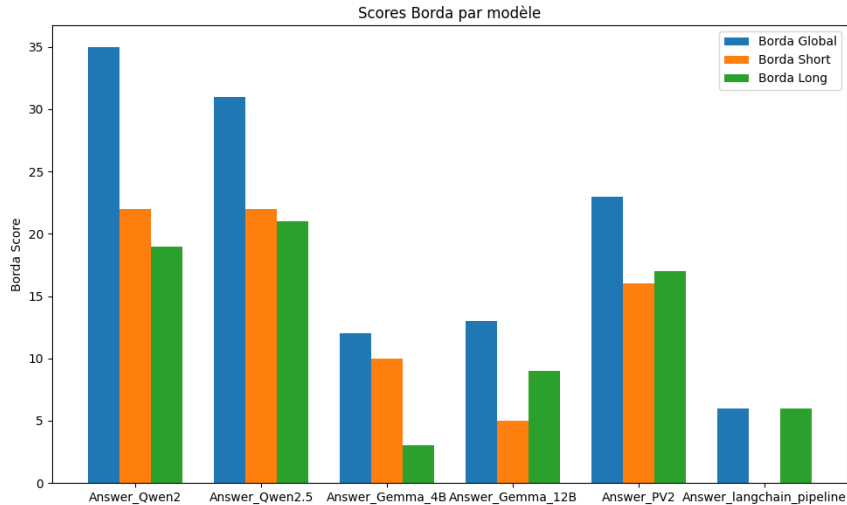


Figure 16: Aggregated Borda Scores by Model

As shown in Figure 18, we consider the model with the highest Borda score to be the best overall performer according to the aggregated metric. Below we provide detailed metric tables for each model:

Model	Borda Global	Faithfulness Avg
Qwen2_2B	35	84.62%
Qwen2.5_7B	31	84.62%
Pipeline V2	23	78.43%
Gemma3_12B	13	82.69%
Gemma3_4B	12	88.46%
Langchain_pipeline	6	65.38%

Table 5: Global Metrics for Generation Evaluation

Model	Faithfulness	Numerical Acc.	ROUGE-1	ROUGE-2	ROUGE-L	String Presence
Gemma3_12B	66.67%	44.44%	21.75%	11.16%	20.17%	27.78%
Gemma3_4B	77.78%	50.00%	23.68%	11.94%	21.74%	33.33%
Pipeline V2	82.35%	61.11%	26.36%	15.16%	23.34%	38.89%
Qwen2_2B	77.78%	55.56%	34.94%	22.14%	32.79%	38.89%
Qwen2.5_7B	88.89%	66.67%	29.45%	17.33%	26.82%	72.22%
Langchain_pipeline	77.78%	44.44%	19.64%	7.87%	18.21%	22.22%

Table 6: Short-Answer Metrics for Generation Evaluation

Model	BERT	Faithfulness	Flan-T5	ROUGE-1	ROUGE-2	ROUGE-L
Gemma3_12B	53.43%	91.18%	6.13%	25.35%	6.03%	16.58%
Gemma3_4B	51.24%	94.12%	5.69%	23.00%	4.02%	14.51%
Pipeline V2	57.76%	76.47%	8.21%	28.66%	9.78%	19.51%
Qwen2_2B	59.09%	88.24%	8.26%	28.45%	8.53%	19.83%
Qwen2.5_7B	58.28%	82.35%	8.17%	29.38%	9.80%	20.43%
Langchain_pipeline	51.65%	58.82%	8.92%	18.42%	3.65%	12.28%

Table 7: Long-Answer Metrics for Generation Evaluation

11 Results and Analysis

Based on the previous results from Tables 5, 6, 7 and Figure 18, it appears that the initial pipeline using ColQwen as the retriever and Qwen2_VL_2B for generation—performs best overall.

However, a closer look at the variation of specific metrics shows that the differences between the two Qwen models are minimal. In particular, the ROUGE scores for both short and long questions, as well as the Flan-T5 and BERT scores for the long questions, display only slight differences. These observations call into question the pertinence of these standard metrics in the context of our use cases and on their coherence to evaluate the best pipeline.

To address this, we implemented a *personal_score* that serves as a ground truth. This score ranges from 0 to 1, where 1 indicates a completely correct and expected answer while 0 indicates no correct information. Based on the coherence of the answer, we adjust the score accordingly.

We then measured the correlation between this personal score and our other evaluation metrics using both the Pearson and Spearman methods. The resulting heatmaps illustrate these correlations:

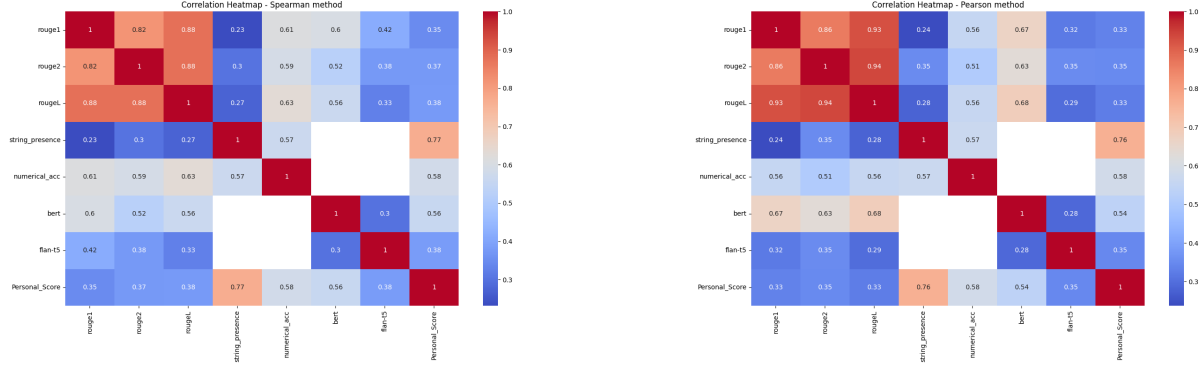


Figure 17: Spearman vs Pearson correlation heatmaps between evaluation metrics and personal score.

The analysis of the heatmaps reveals that the most accurate metric, for our case, is *string_presence* with correlations of approximately 77% and 76% for short-answer questions. This is followed by *numerical_accuracy* and *BERTScore*, which perform reliably on long-answer questions.

Finally, by examining the correlation between the personal score and string presence, we observe a near-linear relationship, which further supports the validity of string presence as a significant metric in our evaluation framework.

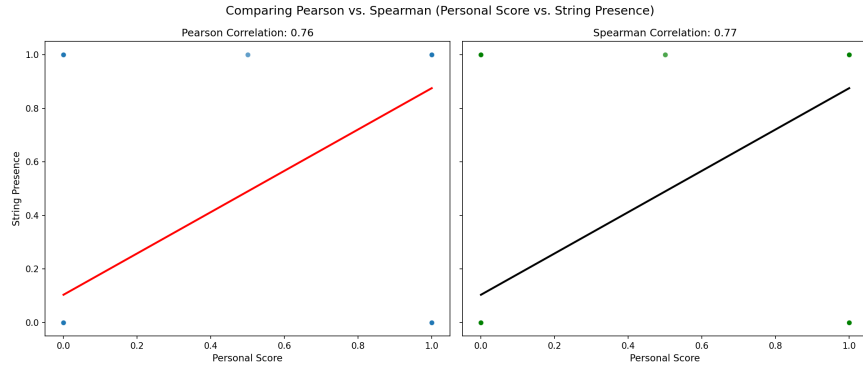


Figure 18: Correlation String Presence Comparison

Based on these specific metrics, the different answers provided by the two Qwen models, and the FinQA performance, it is evident that the **Qwen2.5 7B VL** model offers the best results for our financial question-answering task.

In conclusion, the best pipeline is the initial one—without the additional re-ranking and token-based enhancements—contrary to our initial expectations. This unexpected outcome suggests that, for our dataset and evaluation objectives, the simpler baseline pipeline delivers the most coherent and accurate responses.

12 Challenges Encountered

During this project, we faced **several challenges** espacially during the starting phase that impacted our progress:

1. Data Availability and Future Requirements

While the initial dataset from *Ardian* (see Section 4.3) provided a foundation, we identified the need to source additional, well-labeled, and meaningful datasets to ensure their relevance for future model testing and deployment.

2. Computational Resource Constraints

Our initial LangChain pipeline (see Section 5.1.2) was implemented on *Google Colab*, where limited GPU resources forced us to use smaller, less capable LLMs. This affected both computational performance and graphical processing capabilities. Transitioning to Centrale’s *DCE* introduced further challenges, particularly when integrating ColPali with a Vision-Language Model (VLM), which further delayed progress.

3. Uncertainty in Use Cases

The confirmation of the use cases with *Ardian*, planned for January, required adjustments to technical choices made and also regarding the dataset for evaluation.

4. RAG Methodology Learning Curve

Mastering Retrieval-Augmented Generation (RAG) techniques required time and effort, slightly delaying the initial development phases.

13 Conclusion project

We would like to thank Madame Céline Hudelot, Charles Moslonka, and Hicham Randrianarivo for their great support throughout this project. Their guidance, help in procuring resources, and constant monitoring to guide us and resolve our problem have been of immense use to us.

This project was a very enriching experience and we are truly grateful for this opportunity.

References

- [1] Ragas:library to evaluate llm.
- [2] Zhiyu Chen, Wenhui Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan R Routledge, et al. Finqa: A dataset of numerical reasoning over financial data. *arXiv preprint arXiv:2109.00122*, 2022. Available at <https://arxiv.org/pdf/2109.00122>.
- [3] Manuel Faysse. Analysis of the colpali: Efficient document retrieval with vision language models paper. <https://huggingface.co/blog/manu/colpali>, 2024. Hugging Face Blog, accessed on 2024-12-18.

- [4] Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. Colpali: Efficient document retrieval with vision language models. *arXiv preprint arXiv:2407.01449*, 2024. Available at <https://arxiv.org/abs/2407.01449>.
- [5] N. Bradley Fox, Benjamin Bruyns, et al. Borda count: An evaluation of borda count variations using ranked choice voting data. *arXiv preprint arXiv:2501.00618v2*, 2022. Available at <https://arxiv.org/html/2501.00618v2>.
- [6] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. *arXiv preprint arXiv:2004.12832*, 2020. Available at <https://arxiv.org/abs/2004.12832>.
- [7] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics.
- [8] Chen Ling, Xujiang Zhao, Jiaying Lu, Chengyuan Deng, Can Zheng, Junxiang Wang, Tanmoy Chowdhury, Yun Li, Hejie Cui, Xuchao Zhang, et al. Domain specialization as the key to make large language models disruptive: A comprehensive survey. *arXiv preprint arXiv:2305.18703*, 2023.